Tab 1

🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

A Cisco Networking Academy–style lab that blends classical networking with application-layer engineering. You will configure inter-VLAN routing, deploy a MIME-aware socket server, capture packets, and analyze how meaning flows through the stack.

1.นายปกรณ์เกียรติ ศรีจันทร์ 673380045-2 Sec2

2.นายจิณณวัตร โพธิ์ศรีทอง 673380263-2 Sec2

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client
- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data
- Decode JSON headers in packet payload
- Troubleshoot network and application failures
- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)

- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links
- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

```
None
PC1 (VLAN 10) ----\
                    \ access
                     \
                     [S1]---- trunk ----[R1]
                     /
                    / access
PC2 Server (VLAN 20)
```

📋 Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
None
enable
configure terminal
hostname R1

interface g0/0/0
no shutdown

interface g0/0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0

end
write memory
```

```
Router>en
Router#config t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R1
R1(config)#interface g0/0/0
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

R1(config-if)#interface g0/0/0.10
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0/0.10, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.10, changed state to up

R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#interface g0/0/0.20
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0/0.20, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.20, changed state to up

R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
switchport mode access
switchport access vlan 20

end
```

```
write memory
```

```
Switch>en
Switch#config t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#
```

```
S1(config-vlan)#
S1(config-vlan)#interface g0/1
S1(config-if)#switchport mode trunk

S1(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

S1(config-if)#interface fa0/1
S1(config-if)#switch mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switctport access vlan 20
                ^
% Invalid input detected at '^' marker.

S1(config-if)#end
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#
```

```
S1#write memory
Building configuration...
[OK]
S1#
```

Step 3 — Configure Hosts

Set static IP addresses per table.

🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```

Verify:

- Successful replies
- ARP entries

Router checks:

```
None
show ip route
show ip interface brief
```

Switch checks:

```
None
show interfaces trunk
show mac address-table
```

🧪 Part 3 — Deploy MIME Server

On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port
65432
```

Confirm file saved.

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

Apply capture filter:

```
None
tcp port 65432
```

Step 2 — Initiate Transfer

Send file again.

Stop capture after completion.

Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?

Confirm MAC resolution.

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.

C — Data Transfer

Follow stream:

Right click → Follow → TCP Stream.

You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```

Followed by binary.

D — VLAN Tags

If capturing on trunk:

Look for:

```
None
802.1Q Virtual LAN
```

Note VLAN ID.

🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠 Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment

📝 Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1️⃣ Why does TCP retransmit lost segments?

เพราะ TCP เป็น reliable protocol ถ้าไม่ได้รับ ACK ภายใน timeout จะ retransmit segment เพื่อรับประกันว่า data ครบถ้วน

2️⃣ How does VLAN tagging affect broadcast domains?

VLAN แยก broadcast domain เครื่องใน VLAN 10 จะไม่ broadcast ไป VLAN 20 ลด broadcast traffic และเพิ่ม security

---

3️⃣ What role does MIME play compared to TCP headers?

- TCP header → จัดการ reliability (sequence, ACK, port)
- MIME → ให้ semantic meaning (file type, size)
  TCP ไม่รู้ว่าไฟล์คืออะไรMIME ทำให้ application เข้าใจข้อมูล

4️⃣ Where does routing decision occur?

Routing decision เกิดที่ Router
ใน Network Layer (IP layer)
ดู destination IP แล้วเลือก next hop

5️⃣ Why does ARP precede TCP communication?

ก่อนส่ง TCP segment
ต้องรู้ MAC address ของปลายทางหรือ gateway

ARP ใช้แปลง IP → MAC
จึงต้องเกิดก่อน TCP data transfer

📊 Reflection Exercise

Explain:

How does application meaning survive multiple encapsulation layers?

Application meaning survives because:

1. แต่ละ layer treat upper-layer data เป็น opaque payload
2. ไม่มี layer ใดแก้ไข semantic content
3. TCP รับประกัน reliability
4. Decapsulation คืนข้อมูลเหมือนเดิม

แม้ข้อมูลจะถูก encapsulate หลายชั้น
แต่ meaning ไม่ถูกเปลี่ยน
เพียงแค่ถูกห่อด้วย control information

⚠️ Common Pitfalls

Server bind localhost → client ต่อไม่ได้

Missing trunk → VLAN ข้ามไม่ได้

Wrong VLAN → Ping fail

OS firewall → TCP timeout

Capture wrong interface → ไม่เห็น packet

🚀 Extension Activities

Add TLS encryption

เพิ่มความปลอดภัย
Payload จะถูกเข้ารหัส

Measure throughput vs latency

วัด performance
ดู TCP window effect

Capture checksum errors

ดูความผิดพลาดใน frame

Add checksum field to protocol

เพิ่ม integrity check ใน application layer

Containerize server

ใช้ Docker → ศึกษา distributed system

Compare with HTTP transfer

ดูความแตกต่างระหว่าง custom protocol กับ HTTP

🧬 Instructor Notes — What This Lab Really Teaches
This is not just networking.

1️⃣ Protocol layering philosophy

แต่ละ layer มีหน้าที่ชัดเจน

2️⃣ Reliability engineering

TCP retransmission / flow control

3️⃣ Metadata semantics

MIME ทำให้ข้อมูลมีความหมาย

4️⃣ Distributed system fundamentals

Client ↔ Server ผ่านหลาย layer

TCP ping

Here's how you can modify the script to accept an IP address and port number as command-line arguments using sys.argv:

```python
import sys
from tcppinglib import tcpping
import asyncio

def main():
    # Check if correct number of arguments is provided
    if len(sys.argv) != 3:
        print("Usage: python script.py <IP_ADDRESS> <PORT>")
        sys.exit(1)

    ip_address = sys.argv[1]
    try:
        port = int(sys.argv[2])
    except ValueError:
        print("Error: Port must be an integer.")
        sys.exit(1)

    # Perform TCP ping
    host_stats = tcpping(ip_address, port=port, count=4, interval=1)

    print(f"Host: {host_stats.ip_address}")
    print(f"Is alive: {host_stats.is_alive}")
    print(f"Average RTT: {host_stats.avg_rtt:.2f} ms")
    print(f"Packet loss: {host_stats.packet_loss}%")

if __name__ == "__main__":
    main()
```

Explanation:

· The script uses sys.argv to read command-line arguments.
· It expects two arguments: the IP address (or hostname) and the port number.
· Basic error handling ensures both arguments are provided and the port is a valid integer.
· The tcpping function is called with the provided IP and port, keeping the same count and interval for demonstration.

You can run it like:

```bash
python script.py 192.168.1.1 80
```

If you also want to include the asynchronous version, you can adapt it similarly, but the synchronous example above covers the core requirement.

Role-based procedures

👥 Role-Based Lab Procedure: MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

🎭 Team Roles

Role Primary Responsibilities Suggested Tools
Network Infrastructure Engineer Configures router and switch, sets up VLANs and trunking, assigns IP addresses to router subinterfaces, verifies Layer 2/3 connectivity, troubleshoots network issues. Console access (PuTTY, SecureCRT), Cisco IOS CLI, ping, traceroute, show commands, possibly Packet Tracer for simulation.
Client/Server Engineer Deploys and runs the MIME server and client scripts on the two PCs, initiates file transfers, captures traffic with Wireshark, observes application behavior. Python 3, terminal/command prompt, Wireshark, the provided server/main_enhanced.py and client/main_enhanced.py scripts, a sample file (e.g., sample.png).
Knowledge Manager / Supervisor Oversees the lab workflow, documents configurations and results, leads analysis of packet captures, verifies understanding of concepts, coordinates troubleshooting, compiles the final report. Note-taking app (OneNote, Google Docs), Wireshark (for detailed analysis), lab manual, diagramming tool (draw.io), camera/screenshot tool.

📋 Step-by-Step Procedure

All three team members should be present and collaborate. The Knowledge Manager ensures every step is understood and documented.

Part 1 — Configure Network Devices

Network Infrastructure Engineer

· Access the router (R1) console and configure:
  · Hostname, interface G0/0, subinterfaces G0/0.10 and G0/0.20 with 802.1Q encapsulation and IP addresses per the addressing table.
  · Issue no shutdown on the physical interface.
  · Save configuration with write memory.
· Access the switch (S1) console and configure:
  · VLANs 10 and 20.
  · Interface G0/1 as trunk.
  · Interfaces Fa0/1 (access VLAN 10) and Fa0/2 (access VLAN 20).
  · Save configuration.
· Verify configurations with show ip interface brief, show interfaces trunk, show vlan brief.

Client/Server Engineer

· Not actively involved in this part, but observe the configuration process and take notes of the commands used (with help from the Knowledge Manager).

Knowledge Manager / Supervisor

· Document all configuration commands and their expected effects.
· Take screenshots of the running configurations.
· Ensure that the physical cabling matches the topology (PC1 connected to Fa0/1, PC2 to Fa0/2, router G0/0 to switch G0/1).
· Briefly explain the purpose of trunking and subinterfaces to the team.

Part 2 — Verify Connectivity

Network Infrastructure Engineer

· From the router, ping both PC IP addresses to confirm router-to-host connectivity:
```
ping 192.168.10.10
ping 192.168.20.20
```
· Use show ip route to confirm routes to both subnets are present (directly connected).

Client/Server Engineer

· On PC1, set the static IP address 192.168.10.10/24 with gateway 192.168.10.1.
· On PC2, set the static IP address 192.168.20.20/24 with gateway 192.168.20.1.
· From PC1, ping PC2:
```
ping 192.168.20.20
```
· If pings fail, work with the Network Infrastructure Engineer to troubleshoot (check VLAN assignment, trunk, IP settings).

Knowledge Manager / Supervisor

· Record successful ping results (screenshot).
· Verify that ARP entries are populated on both PCs (arp -a).
· Ensure the team understands why ping works across VLANs (router-on-a-stick provides inter-VLAN routing).

Part 3 — Deploy MIME Server and Transfer a File

Client/Server Engineer

· On PC2 (server) , open a terminal and start the server:
```bash
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```
· On PC1 (client) , open another terminal and send a file:
```bash
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port 65432
```
· Confirm that the file is saved on the server side (e.g., check the destination folder).
· If the transfer fails, verify that the server is listening on 0.0.0.0 (not localhost) and that no firewall is blocking port 65432.

Network Infrastructure Engineer

· Monitor network statistics during transfer (optional, e.g., show interface counters).
· Assist in troubleshooting connectivity if needed.

Knowledge Manager / Supervisor

· Document the exact commands used and their output.
· Ensure the team understands the MIME header concept (JSON metadata + binary payload).
· Take a screenshot of the server log showing the received file information.

Part 4 — Wireshark Capture and Analysis

Client/Server Engineer (both PC1 and PC2)

· On both PCs, launch Wireshark and start capturing on the network interface connected to the switch.
· Apply a capture filter to isolate the traffic:
```
tcp port 65432
```
· On PC1, re-run the file transfer command (the client).
· Stop the capture on both machines after the transfer completes.
· Save the capture files (e.g., client_capture.pcapng, server_capture.pcapng).

Knowledge Manager / Supervisor

· Lead the analysis of the captured packets.
  · On PC1's capture, locate:
    · ARP request/reply for the server's IP (if not already cached).
    · TCP three-way handshake (SYN, SYN-ACK, ACK).

· Data packets carrying the JSON header (right-click → Follow → TCP Stream to see the full conversation).
· The binary file data.
· On PC2's capture, compare the same packets from the server's perspective.
· If capturing on the trunk link (optional), look for 802.1Q tags with VLAN IDs 10 and 20.
· For each layer (Ethernet, 802.1Q, IP, TCP, Application), note the fields listed in the dissection guide.
· Take screenshots of:
· ARP packet details.
· TCP handshake packets.
· The JSON header inside the TCP stream.
· A packet containing binary payload.
· (Optional) VLAN tag information.

Network Infrastructure Engineer

· Assist in interpreting VLAN tags if trunk capture was performed.
· Explain how the router decides to forward packets based on destination IP and VLAN.

Part 5 — Troubleshooting Scenarios

The team works together to simulate failures and observe the effects. The Knowledge Manager predicts the expected behavior, then the group implements the fault and observes.

Scenario Action by … Expected Observation Documented Result
Broken Routing Network Infrastructure Engineer shuts down subinterface G0/0.10 on the router. Ping from PC1 to PC2 fails; TCP SYN from client times out (retransmissions). Screenshot of failed ping and Wireshark showing SYN retransmits.
Firewall Block Client/Server Engineer (on server) adds a firewall rule to block port 65432. Client's TCP SYN is unanswered; Wireshark shows repeated SYN attempts. Screenshot of firewall rule and capture showing no SYN-ACK.
Wrong Gateway Client/Server Engineer changes PC1's gateway to an incorrect IP. Ping fails; ARP requests for the gateway remain unanswered ("ARP incomplete"). Screenshot of arp -a showing incomplete entry.
Packet Loss Network Infrastructure Engineer uses a traffic control tool (e.g., tc on Linux) to drop 10% of packets. TCP retransmissions and possible window size adjustment; file still transfers reliably. Wireshark capture showing duplicate ACKs and retransmissions.

Knowledge Manager / Supervisor records each observation and leads a discussion on why TCP behaves the way it does.

📝 Lab Report Compilation

The Knowledge Manager is responsible for assembling the final lab report, incorporating:

· Screenshots from all parts (configs, ping results, Wireshark captures, JSON header).
· Explanations of each observation, linking to OSI layers.
· Answers to the assessment questions.
· A reflection on how application-layer meaning survives encapsulation.

The Network Infrastructure Engineer and Client/Server Engineer provide their notes, captures, and any troubleshooting logs.

✅ Collaboration Checkpoints

At the end of each major part, the team should pause and:

· Review what was accomplished.
· Ensure all documentation (screenshots, notes) is saved.
· Discuss any unexpected issues and how they were resolved.

This role-based approach ensures every student actively contributes while learning both the technical and analytical aspects of the lab.